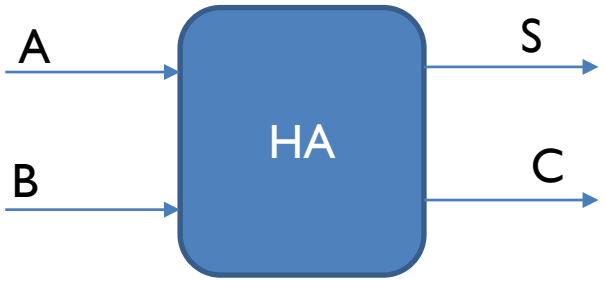
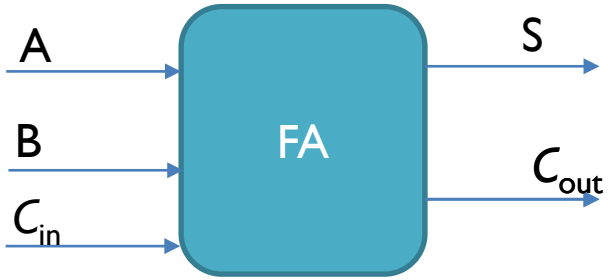
Lecture 6: Finite State Machine, Electronic Door Lock (EDL), and LCD [Hints on SPI]

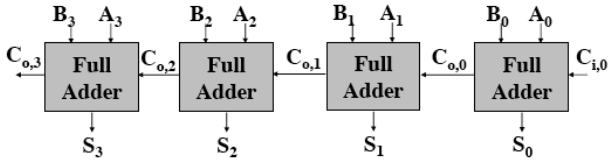
By: Emad Samuel Malki Ebeid

Summary of lecture 5

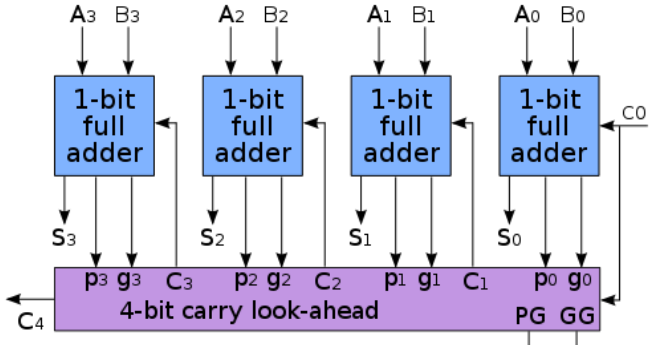




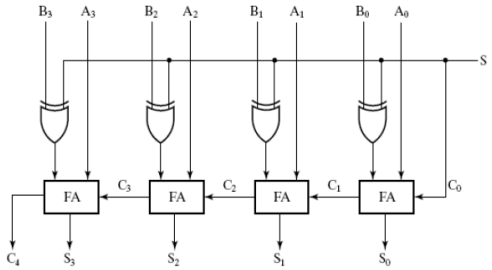
- **Ripple-carry adder**



- **Carry-look-ahead adder**

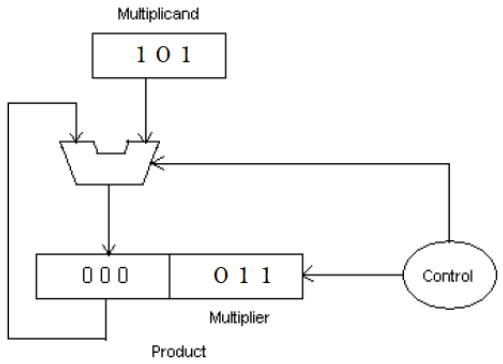


- Full Adder/Subtractors

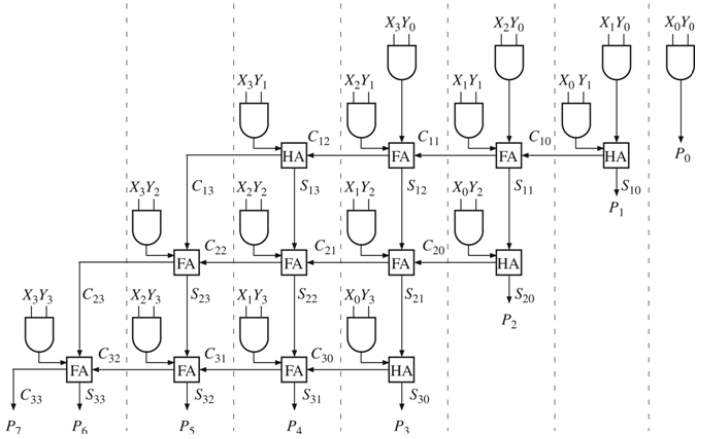


Summary of lecture 5

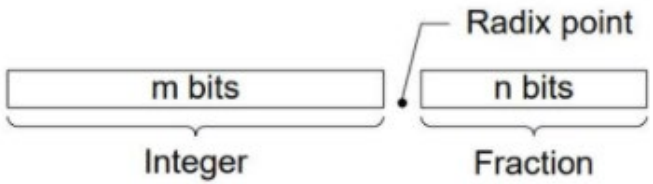
- Multiplier



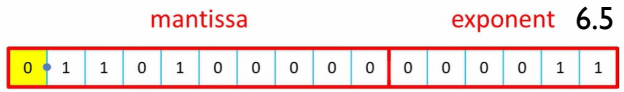
- Array Multiplier



- Dividers
  - Clk divider
  - Division by subtraction
  - Division by multiplication
- Fixed point



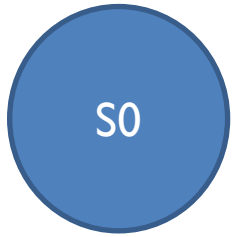
- Floating point



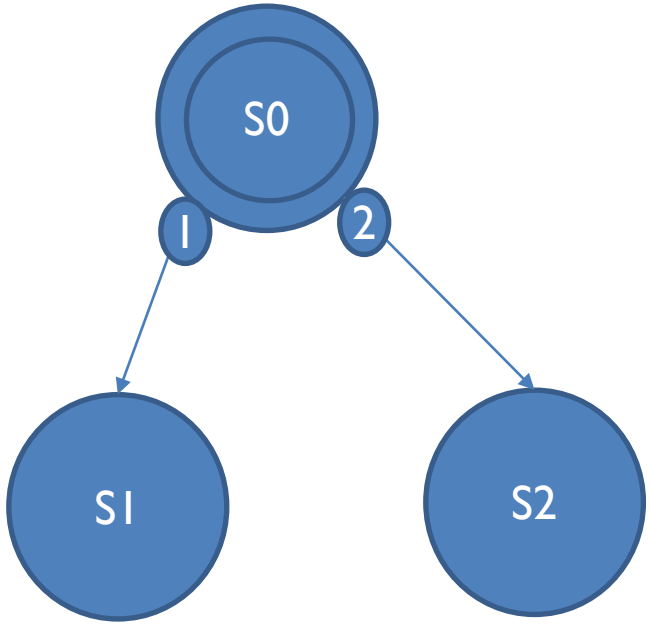
Finite State Machine (FSM)

- FSMs are mathematical abstractions that are used to solve a large variety of problems, among which are electronic design automation, communication protocol design, parsing and other engineering applications.
- It is a step beyond the sequential circuit design.

- FSM consists of:
  - States

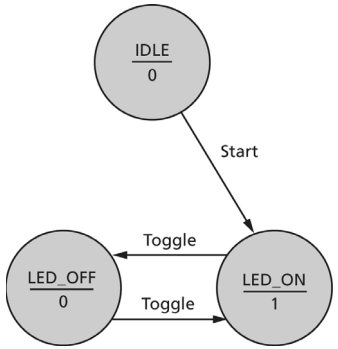


  - Transitions



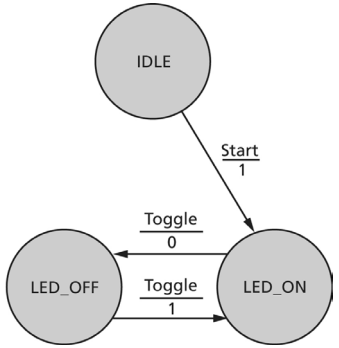
FSM

Moore State Machine



The outputs of a Moore state machine depend **only on the present state**. The <u>outputs</u> are written **only** when the state changes <u>(on the clock edge)</u>.

Mealy State Machine

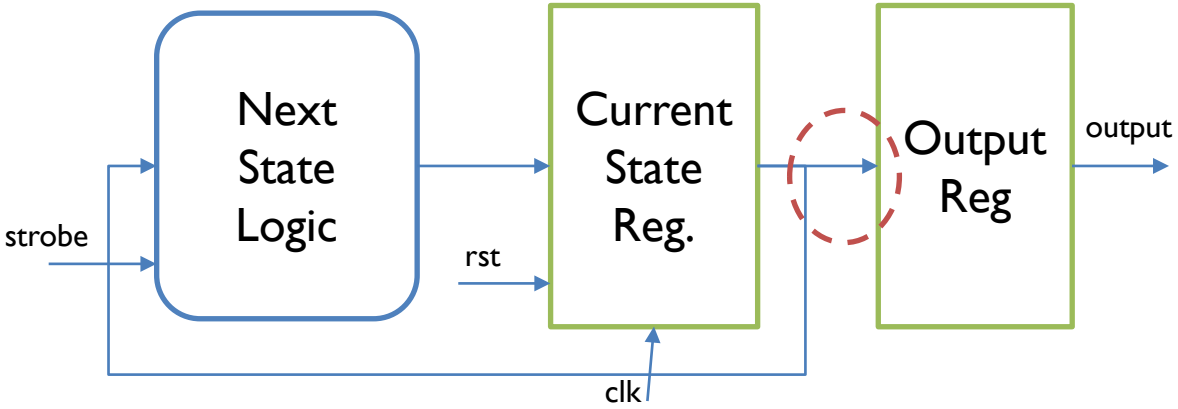


The outputs of a Mealy state machine depend on **both the inputs and the current state**. When the <u>inputs</u> change, the outputs are updated <u>**without** waiting for a clock edge</u>.

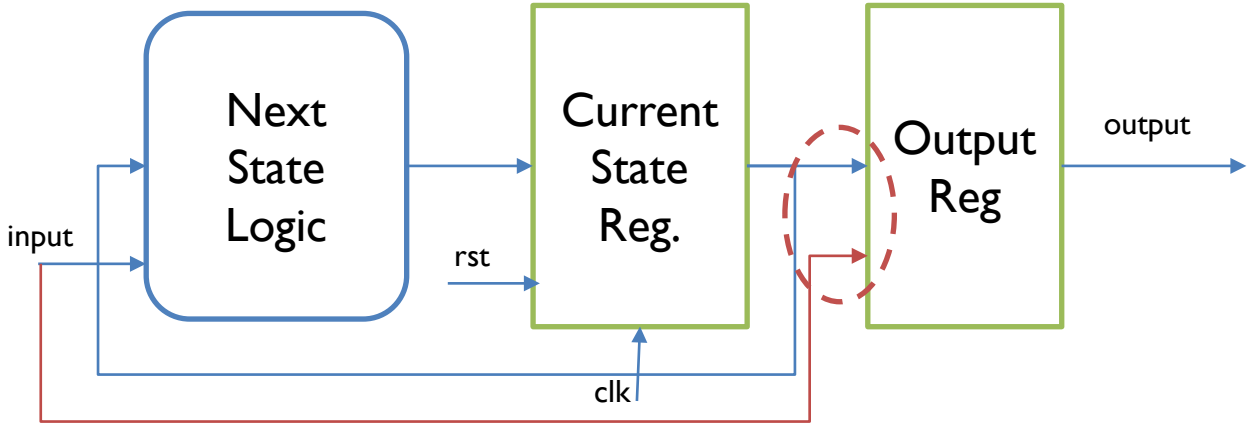
Because modern designs are generally synchronous, the **Moore** option tends to be preferred – *Finite State Machine in Hardware* book by Volnei A. Pedroni

FSM in hardware

Moore

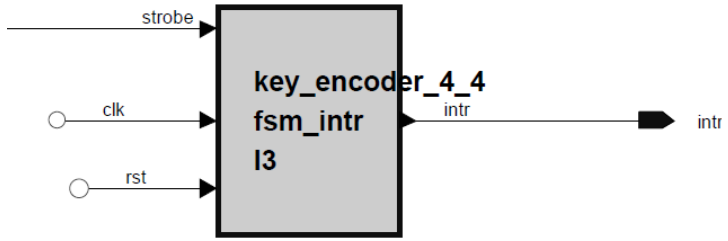


Mealy

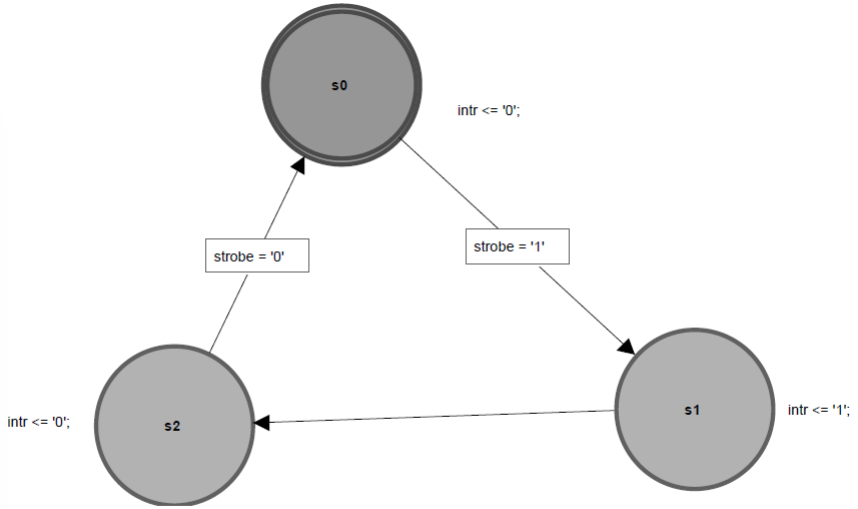


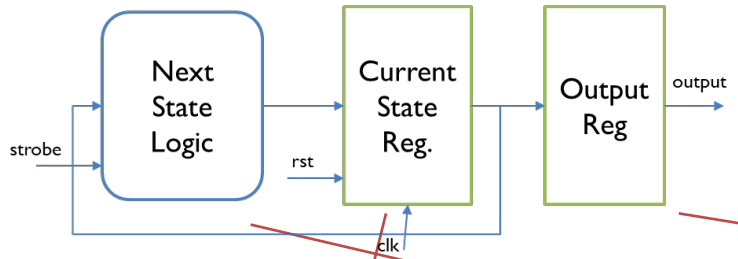
FSM examples: debouncer

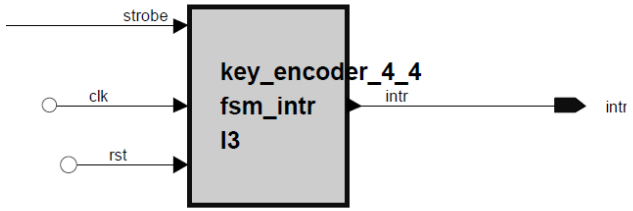
```vhdl
ENTITY fsm_intr IS
   PORT(
      clk    : IN     std_logic;
      rst    : IN     std_logic;
      strobe : IN     std_logic;
      intr   : OUT    std_logic
   );

-- Declarations

END fsm_intr ;
```

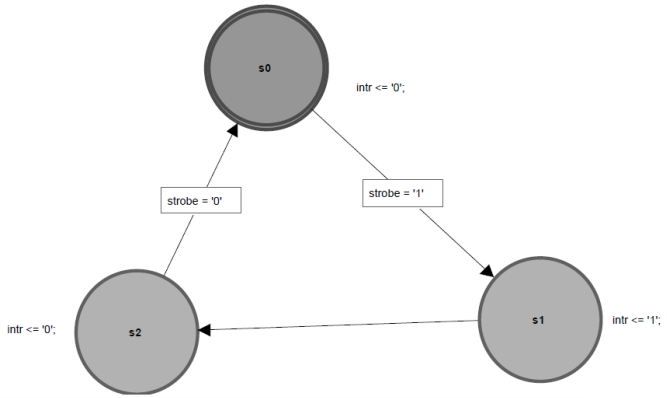



```vhdl
ARCHITECTURE fsm OF fsm_intr IS

   -- Architecture Declarations
   TYPE STATE_TYPE IS (s0, s1, s2); -- Define the states




   -- Declare current and next state signals
   SIGNAL current_state : STATE_TYPE ; -- Create a signal that uses the different states
   SIGNAL next_state : STATE_TYPE ;

BEGIN
```









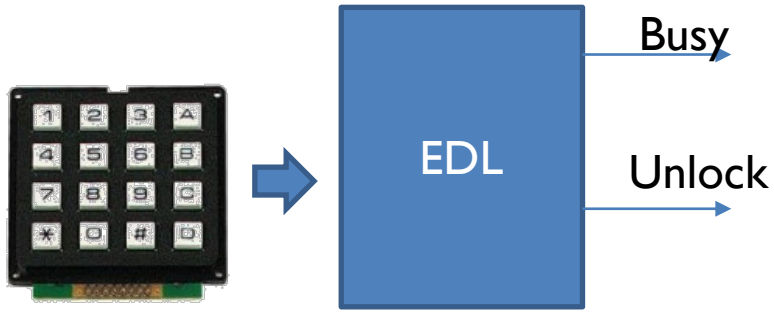

```
-----------------------------------------------------------------
clocked : PROCESS(clk, rst)
-----------------------------------------------------------------
BEGIN
   IF (rst = '1') THEN
      current_state <= s0;
      -- Reset Values
   ELSIF (rising_edge(clk)) THEN
      current_state <= next_state;
      -- Default Assignment To Internals

      -- Combined Actions for internal signals only
--    CASE current_state IS
--    WHEN s0 =>
--       reg<= data_in & '1';
--    WHEN s2 =>
--       count<= count +1;
--    WHEN OTHERS =>
--       NULL;
--   END CASE;
   END IF;

END PROCESS clocked;
```

```
-----------------------------------------------------------------
nextstate : PROCESS (current_state, strobe)
-----------------------------------------------------------------
BEGIN
   CASE current_state IS
   WHEN s0 =>
      IF (strobe = '1') THEN
         next_state <= s1;
      ELSE
         next_state <= s0;
      END IF;
   WHEN s1 =>
         next_state <= s2;
   WHEN s2 =>
      IF (strobe = '0') THEN --IF (count = 10) THEN
         next_state <= s0;
      ELSE
         next_state <= s2;
      END IF;
   WHEN OTHERS =>
      next_state <= s0;
   END CASE;

END PROCESS nextstate;
```

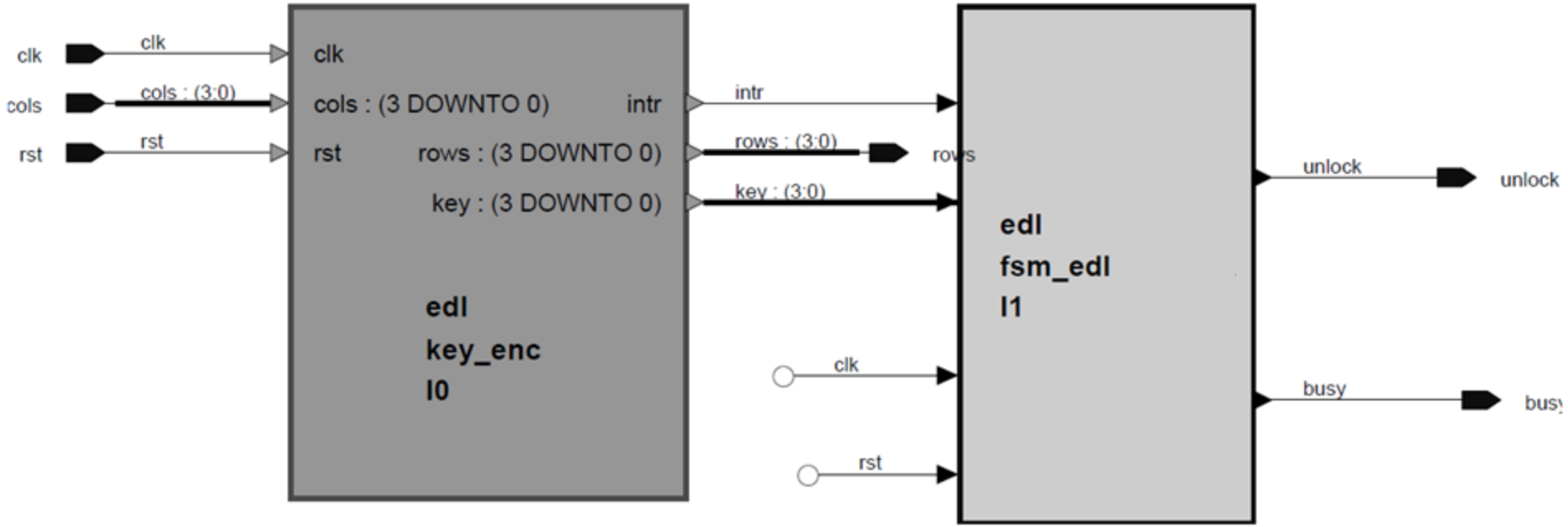
```
   -----------------------------------------------------------------
   output : PROCESS (current_state)
   -----------------------------------------------------------------
   BEGIN
      -- Default Assignment
      intr <= '0';
      -- Default Assignment To Internals

      -- Combined Actions
      CASE current_state IS
      WHEN s0 =>
         intr <= '0';
      WHEN s1 =>
         intr <= '1';
      WHEN s2 =>
         intr <= '0';
      WHEN OTHERS =>
         NULL;
      END CASE;

   END PROCESS output;

   -- Concurrent Statements

END fsm;
```

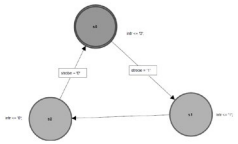
Electronic Door Lock (EDL)

- It is an electronic circuit that opens a lock when a PIN code is entered correctly.
- It usually has a keypad as an interface and two output signals (busy and unlock)

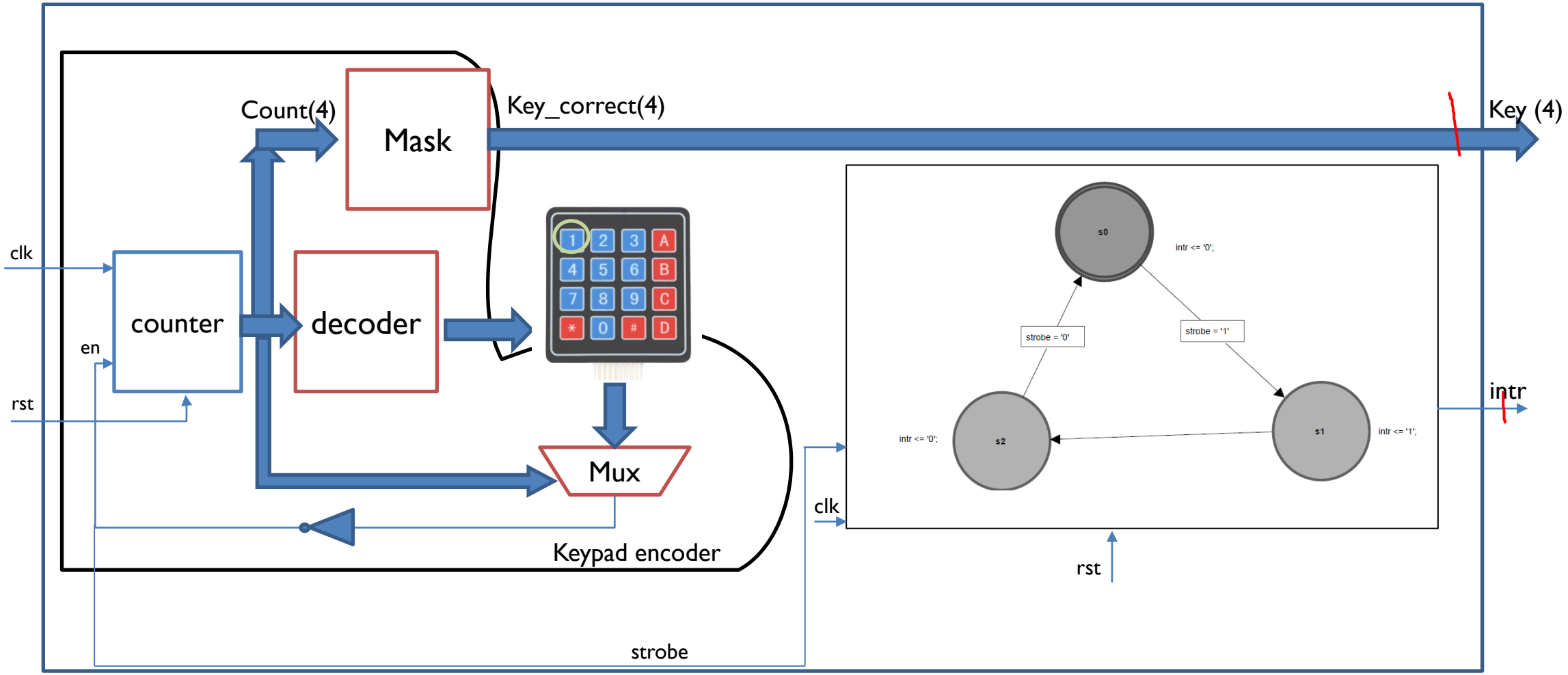


EDL block diagram

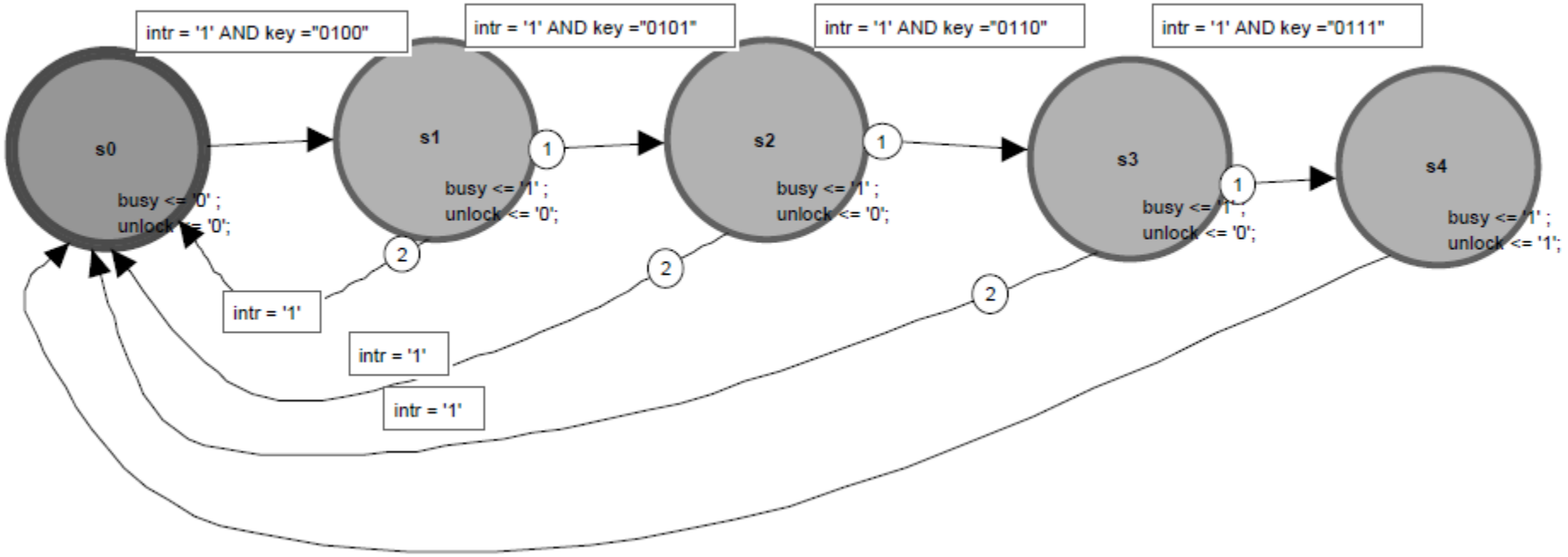




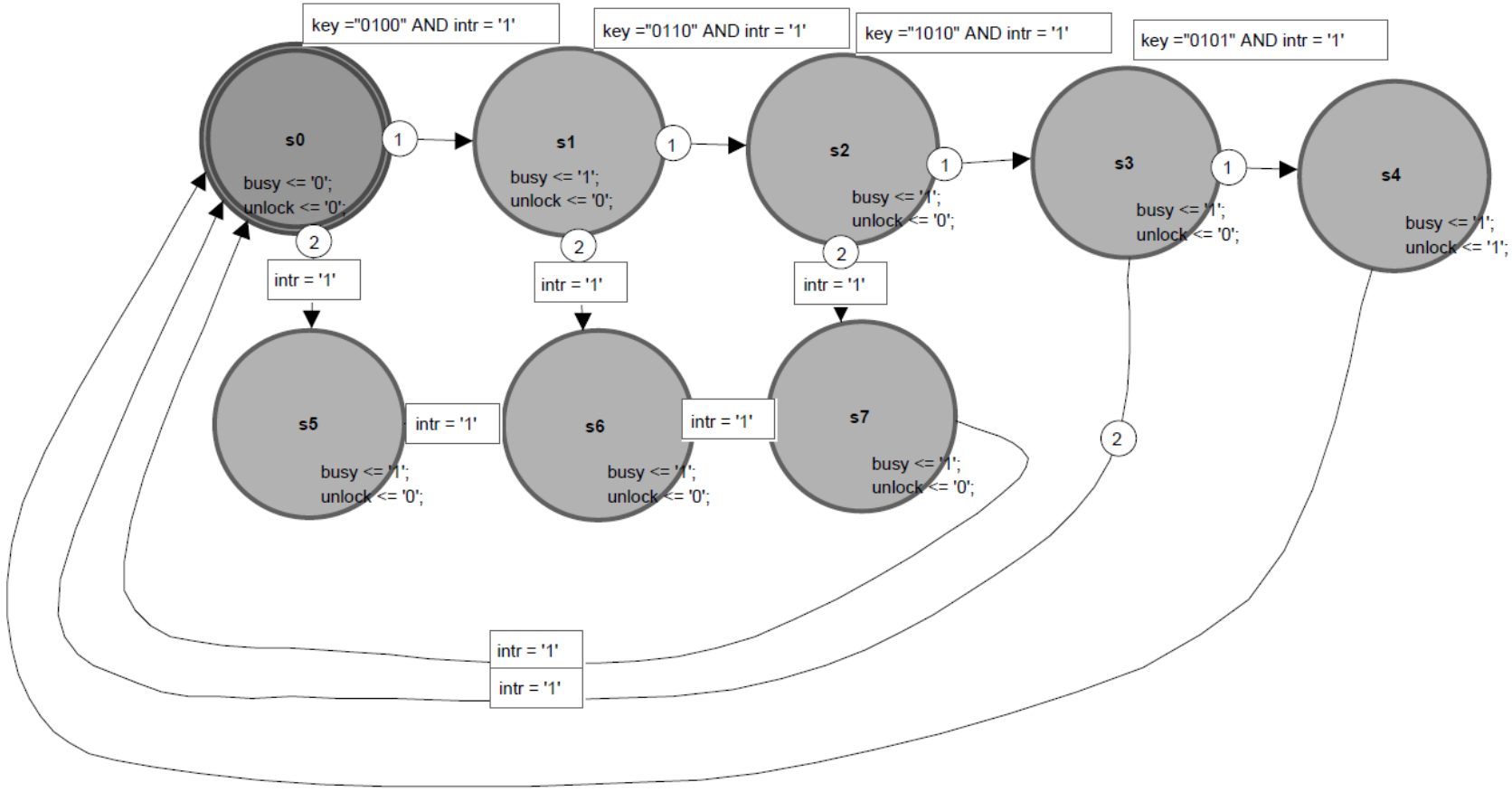
Keypad encoder with interrupt



FSM simplex

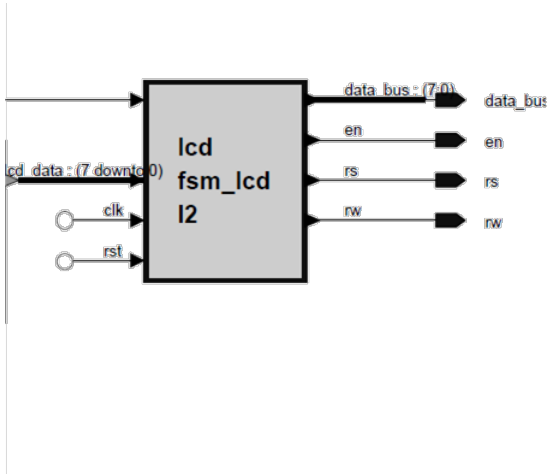

intr = '1' AND key ="0100"
intr = '1' AND key ="0101"
intr = '1' AND key ="0110"
intr = '1' AND key ="0111"
s0
busy <= '0' ;
unlock <= '0';
s1
busy <= '1' ;
unlock <= '0';
s2
busy <= '1' ;
unlock <= '0';
s3
busy <= '1' ;
unlock <= '0';
s4
busy <= '1' ;
unlock <= '1';
1
1
1
2
2
2
intr = '1'
intr = '1'
intr = '1'

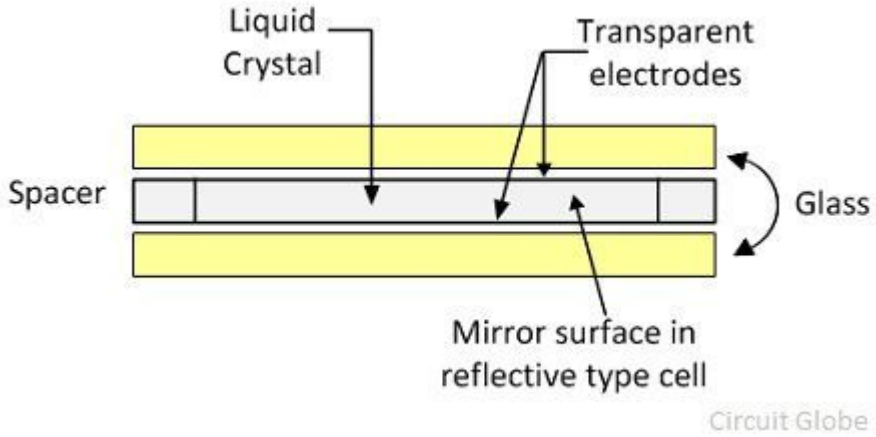
FSM complex


key ="0100" AND intr = '1'
key ="0110" AND intr = '1'
key ="1010" AND intr = '1'
key ="0101" AND intr = '1'
s0
busy <= '0';
unlock <= '0';
s1
busy <= '1';
unlock <= '0';
s2
busy <= '1';
unlock <= '0';
s3
busy <= '1';
unlock <= '0';
s4
busy <= '1';
unlock <= '1';
1
2
intr = '1'
s5
busy <= '1';
unlock <= '0';
s6
busy <= '1';
unlock <= '0';
s7
busy <= '1';
unlock <= '0';

Liquid-Crystal Display (LCD)

- **LCD** is a flat panel display technology commonly used in calculators, TVs and computer monitors.





LCD datasheet

(The datasheet is in the lecture 6 fiels)

Control command

Table 2. The Command Control Codes.

Command	Binary D7	D6	D5	D4	D3	D2	D1	D0	Hex
Clear Display	0	0	0	0	0	0	0	1	01
Display & Cursor Home	0	0	0	0	0	0	1	x	02 or 03
Character Entry Mode	0	0	0	0	0	1	I/D	S	04 to 07
Display On/Off & Cursor	0	0	0	0	1	D	U	B	08 to 0F
Display/Cursor Shift	0	0	0	1	D/C	R/L	x	x	10 to 1F
Function Set	0	0	1	8/4	2/1	10/7	x	x	20 to 3F
Set CGRAM Address	0	1	A	A	A	A	A	A	40 to 7F
Set Display Address	1	A	A	A	A	A	A	A	80 to FF

I/D: 1=Increment*, 0=Decrement
S: 1=Display shift on, 0=Display shift off*
D: 1=Display On, 0=Display Off*
U: 1=Cursor underline on, 0=Underline off*
B: 1=Cursor blink on, 0=Cursor blink off*
D/C: 1=Display shift, 0=Cursor move

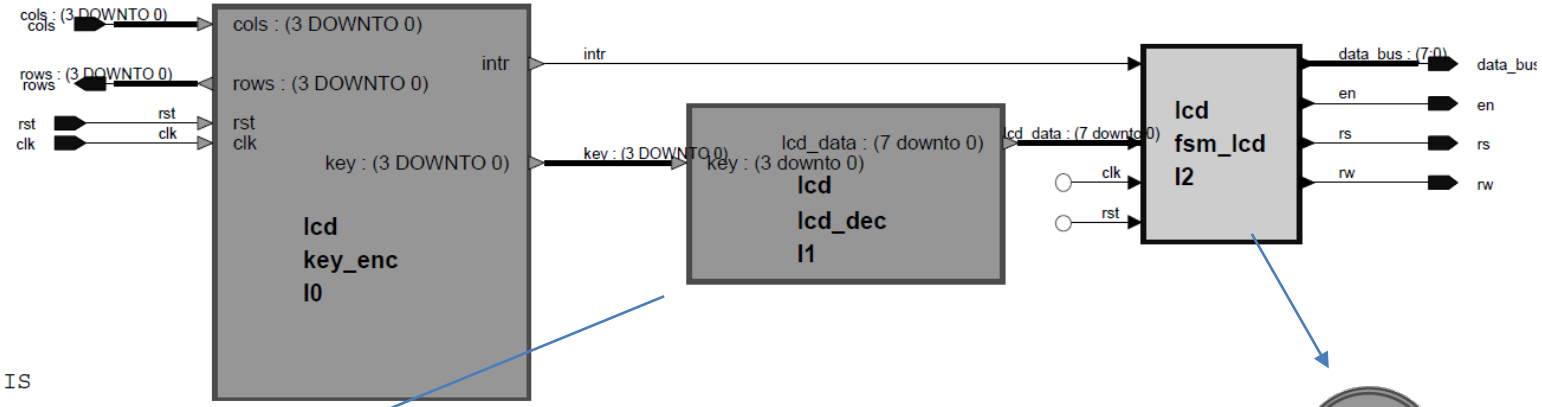
R/L: 1=Right shift, 0=Left shift
8/4: 1=8 bit interface*, 0=4 bit interface
2/1: 1=2 line mode, 0=1 line mode*
10/7: 1=5x10 dot format, 0=5x7 dot format*

x = Don't care * = Initialisation settings

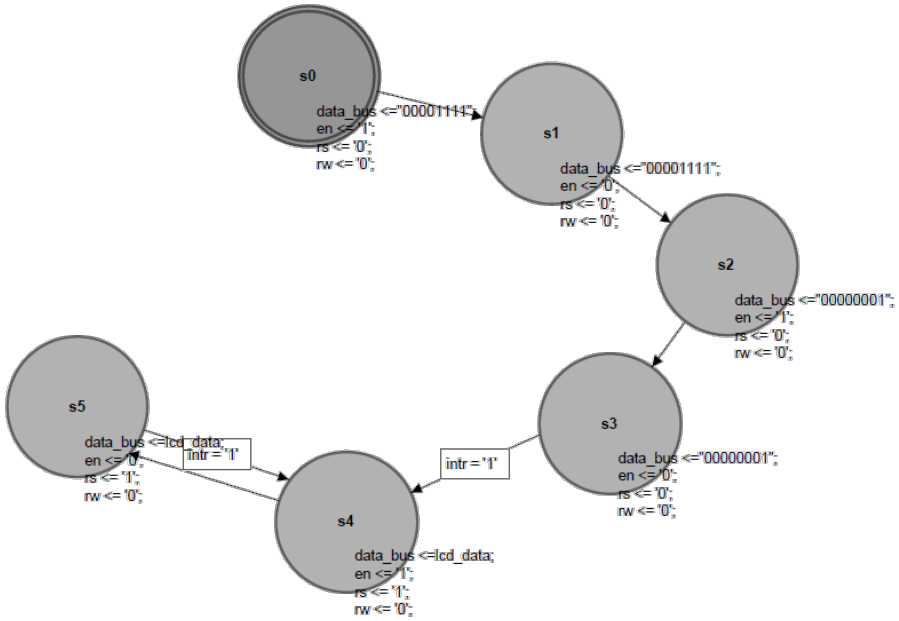
Data to be displayed

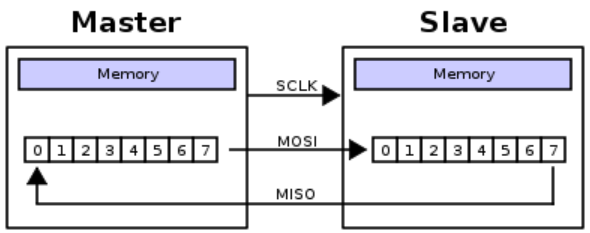
Table 3. Standard l.c.d. character table.

LCD driver



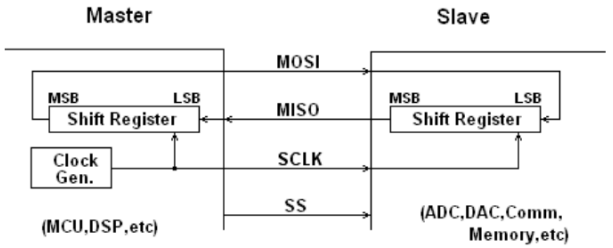

```
-- hds interface_end
ARCHITECTURE rtl OF lcd_dec IS
BEGIN
process(key)
begin
case key is
when "0000"=>lcd_data<="00110000";
when "0001"=>lcd_data<="00110001";
when "0010"=>lcd_data<="00110010";
when "0011"=>lcd_data<="00110011";
when "0100"=>lcd_data<="00110100";
when "0101"=>lcd_data<="00110101";
when "0110"=>lcd_data<="00110110";
when "0111"=>lcd_data<="00110111";
when "1000"=>lcd_data<="00111000";
when "1001"=>lcd_data<="00111001";
when "1010"=>lcd_data<="01000001";
when "1011"=>lcd_data<="01000010";
when "1100"=>lcd_data<="01000011";
when "1101"=>lcd_data<="01000100";
when "1110"=>lcd_data<="01000101";
when others=>lcd_data<="01000110";
end case;
end process;

END rtl;
```

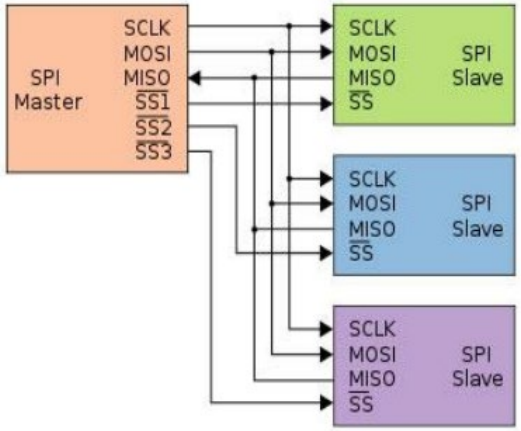




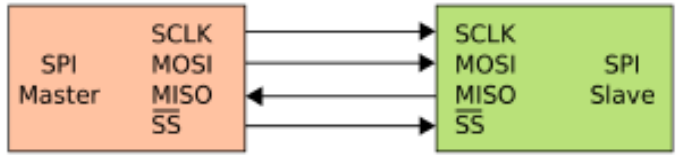
SPI

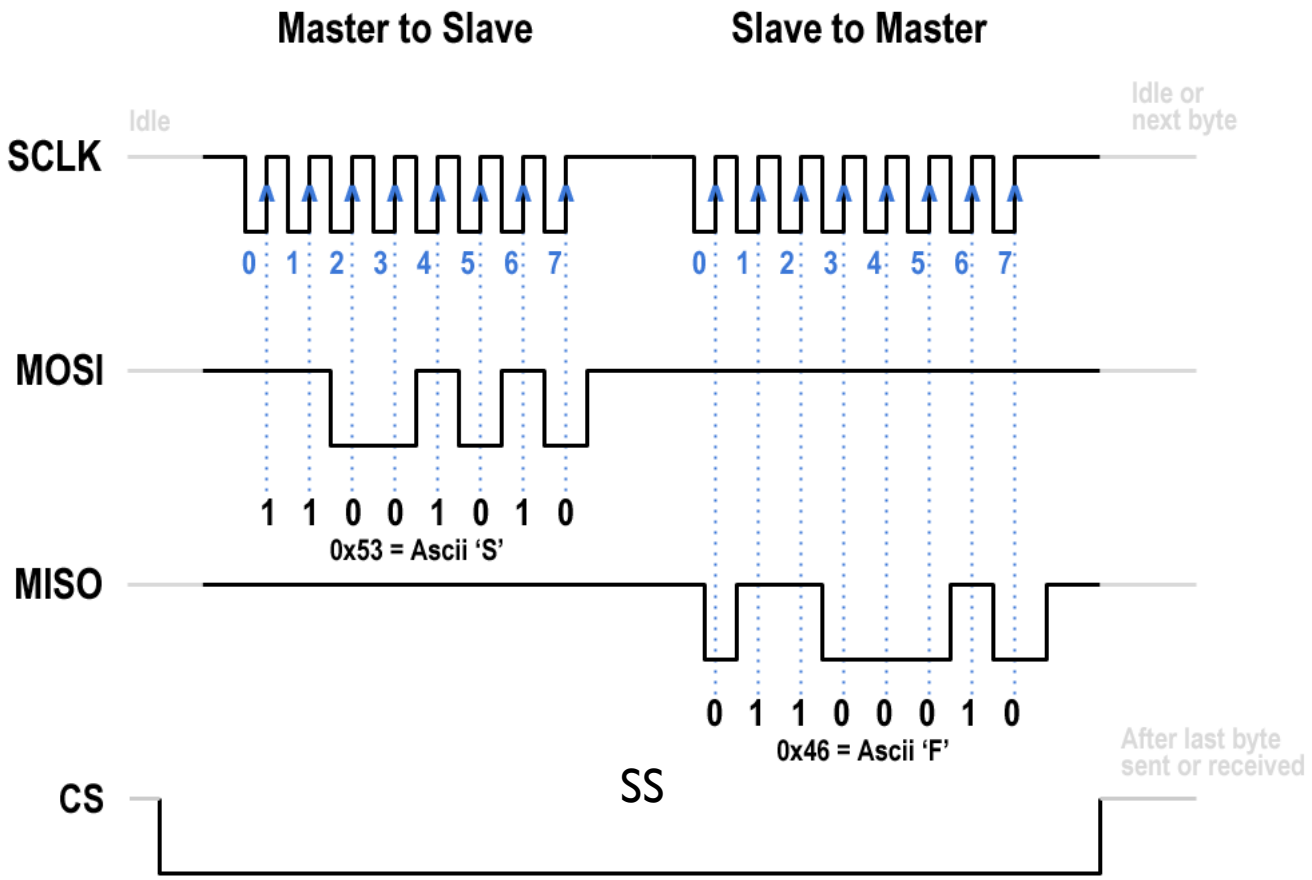
- SPI works by shifting bits between two registers of the same word size. Data is exchanged through the two connections, MOSI and MISO.
- One bit is shifted for each cycle of the serial clock.
- This yields a constant transfer rate. The first bit delivered is the most significant bit and the least significant bit is the last.
  - SCLK is the serial clock.
  - MOSI is the output from the master to the slave.
  - MISO is the output from the slave to the master.
  - SS is the slave select, which is used to wake up the slave.



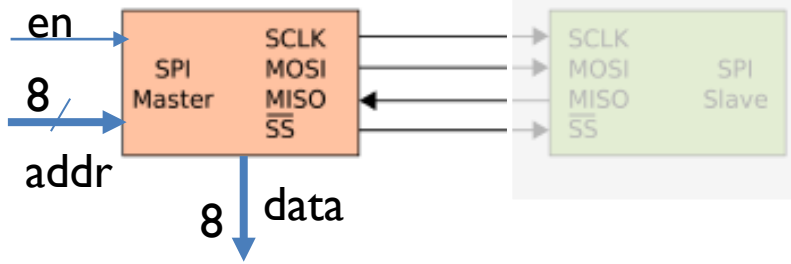
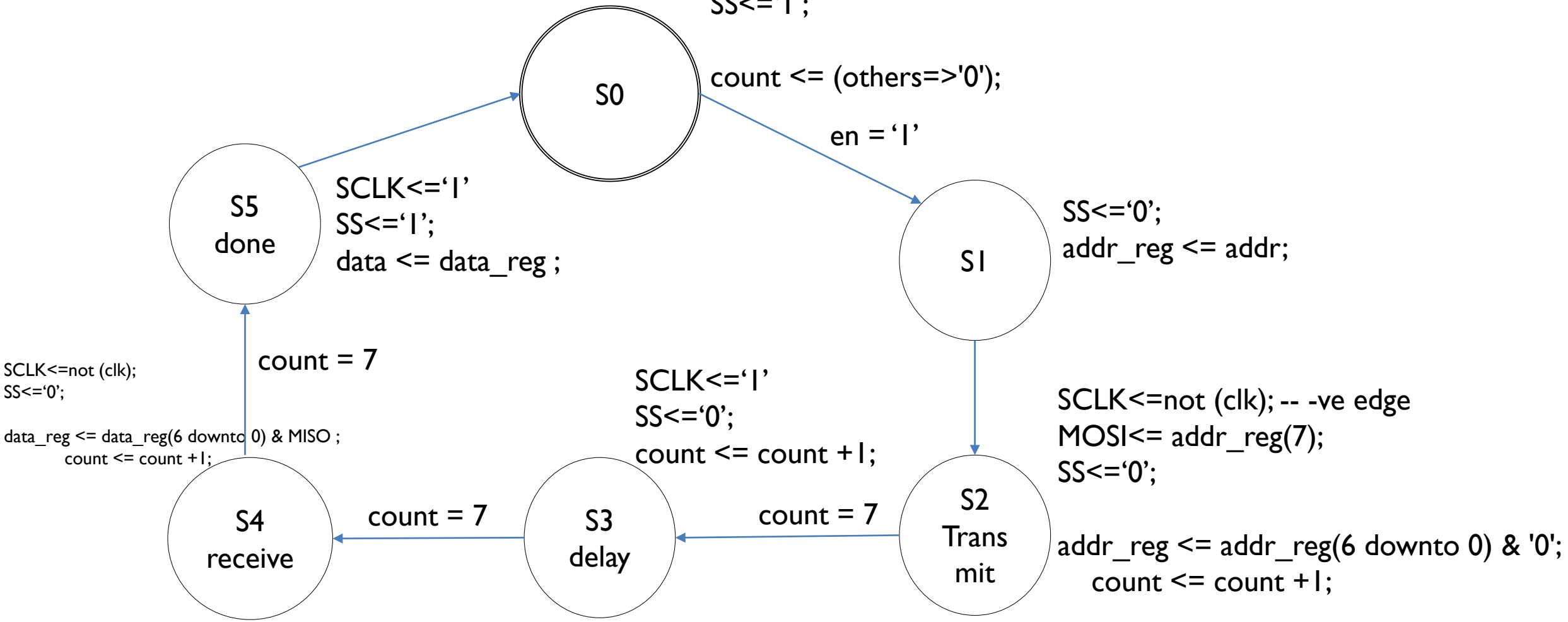


How it works


SPI
Master
SCLK
MOSI
MISO
SS
SCLK
MOSI
MISO
SS
SPI
Slave


Master to Slave
Slave to Master
Idle
Idle or
next byte
SCLK
0 1 2 3 4 5 6 7
0 1 2 3 4 5 6 7
MOSI
1 1 0 0 1 0 1 0
0x53 = Ascii 'S'
MISO
0 1 1 0 0 0 1 0
0x46 = Ascii 'F'
After last byte
sent or received
CS
SS

Internal signals:

addr_reg, data_reg: std_logic_vector(7 downto 0) ;
count: unsigned(2 downto 0) ;

SPI FSM: Master